ratatui rust benzeri ve fakat gnu assembler c ve C++ ile yazılmış python ile script dili olan guile scheme lisp ve emacs lisp ile düzenlenebilen emacs benzeri bir terminal user interface ve amaca yönelik framework yazmak istiyorum

Harika ve oldukça iddialı bir sistem mimarisi hedefliyorsunuz. **Ratatui gibi performansı yüksek, Emacs gibi Lisp dilleriyle (Guile Scheme/Elisp) dinamik olarak genişletilebilen ve çekirdeği GNU Assembler/C/C++ ile yazılmış bir Terminal User Interface (TUI) framework'ü kesinlikle inşa edilebilir.**

Böyle bir projeyi hayata geçirmek için katmanlı bir mimari kurmanız gerekir. Aşağıda, projenizin temel yapı taşlarını ve bu bileşenlerin birbiriyle nasıl konuşacağını gösteren mimari rehberi bulabilirsiniz.

🏛️ Sistem Mimarisi ve Katmanlar

Projeniz temelde **3 ana katmandan** oluşmalıdır:

1. Alt Seviye ve Performans Katmanı (GNU Assembler - GAS)

Her şeyi Assembly ile yazmak sürdürülebilir değildir, ancak **Ratatui**'nin en güçlü olduğu yer olan "Virtual DOM / Çift Tamponlama (Double Buffering)" mekanizmasını Assembly ile optimize edebilirsiniz.

-
- **Kullanım Alanı:** Terminal ekranındaki eski matris (hücreler) ile yeni matrisi karşılaştıran (Diffing) algoritma, terminal kaçış dizilerini (escape sequences) en optimize şekilde string olarak birleştiren döngüler ve memcpy / memset türevi bellek operasyonları.
- **Entegrasyon:** GAS ile yazdığınız fonksiyonları C sembolleri (.global my_asm_func) olarak dışa aktarıp, C/C++ tarafında extern "C" ile çağırabilirsiniz.
-

2. Çekirdek ve Framework Katmanı (C ve C++)

TUI'nin ana motoru, widget ağacı ve terminal durum yönetimi burada yaşar.

-
- **C++20/23 Kullanımı:** Widget hiyerarşisi, Text hizalama (Layout engine - Flexbox benzeri), ioctl ile terminal boyutu değişimlerini (SIGWINCH) yakalama ve ANSI escape kodlarının üretimi.
- **Emacs Benzeri Buffer Yönetimi:** Tıpkı Emacs'teki gibi her pencereyi (Window) bir Buffer nesnesine bağlayan, metinleri "Gap Buffer" veya "Rope" veri yapısıyla tutan mekanizmayı C++ ile kurmalısınız.
-

3. Betikleme ve Düzenleme Katmanı (Python, Guile Scheme, Elisp)

Kullanıcının (veya sizin) TUI uygulamasını run-time'da hackleyebilmesi, yeni tuş kombinasyonları ve temalar ekleyebilmesi için bu katman şarttır.

-
- **Guile Scheme Entegrasyonu:** GNU projesinin resmi uzantı dilidir. libguile C kütüphanesini C++ projenize dahil ederek, C++ fonksiyonlarını Scheme dünyasına bind edebilirsiniz. (Emacs benzeri bir yapı için en ideal dildir).
- **Emacs Lisp (Elisp) Alternatifi:** Gerçek Elisp'i sıfırdan gömmek zordur, ancak Guile Scheme üzerinde çalışan Elisp emülasyon katmanlarını inceleyebilir veya minimal bir Lisp yorumlayıcısı (S-Expression parser) yazabilirsiniz.
- **Python Entegrasyonu:** pybind11 kullanarak C++ çekirdeğinizi Python'a bir modül olarak sunabilirsiniz.
-

⚙️ Yol Haritası ve İlk Adımlar

Projeye başlarken her şeyi aynı anda kurmak yerine **aşağıdaki sırayı** takip etmeniz işinizi kolaylaştıracaktır:

1. **Termios ve Raw Mode:** C/C++ ile terminalin "Raw Mode" özelliğini açın. Standart girdiyi (klavye) anlık okuma ve terminal boyutlarını alma fonksiyonlarını yazın.
2. **Çift Tamponlu Ekran Matrisi:** Bellekte Hücre (Cell) adında bir struct tanımlayın (Karakter, Ön Plan Rengi, Arka Plan Rengi, Modifikatörler). Ekran boyutunda iki adet Cell* matrisi oluşturun (Mevcut Ekran ve Yeni Ekran).
3. **Assembly Diff Motoru:** İşte GAS'ı devreye alma vakti! İki matrisi karşılaştırıp sadece "değişen" hücreleri bulan ve bunları ekrana basacak ANSI koduna dönüştüren döngüyü Assembly ile yazın.
4. **Guile C API Entegrasyonu:** Projeye libguile ekleyin. Örneğin Scheme tarafında (define-key "C-x C-c" 'quit) yazıldığında C++ tarafındaki kapatma fonksiyonunun tetiklenmesini sağlayın.

🛠️ İlham Alabileceğiniz Mevcut Projeler

Sıfırdan tekerleği icat etmemek (veya nasıl yapıldığını görmek) için şu açık kaynak kodlu projeleri mutlaka inceleyin:

-
- **GNU Guile:** C projesine Lisp/Scheme dünyasını nasıl gömeceğinizi anlamak için resmi dökümantasyonu inceleyin.
- **Notcurses:** C ile yazılmış, inanılmaz hızlı ve modern bir TUI kütüphanesidir. Assembly seviyesinde optimizasyonlar ve matrix mantığı için harika bir referanstır.
- **Ncurses:** Klasik terminal yönetim standartlarını anlamak için kaynak kodlarına göz atabilirsiniz.
-

Bu sistemi prototiplemeye başlamak için ilk olarak hangi katmandan (Lisp entegrasyonu mu yoksa Assembly ekran motoru mu) başlamayı düşünüyorsunuz?

Projeye ilk adımı atmak için:

- Çekirdeği **C++** ile kurup **Assembly** ile mi optimize etmek istersiniz, yoksa saf **C** mimarisi üzerinden mi ilerlemek istersiniz?
- Lisp motoru olarak doğrudan **GNU Guile** kütüphanesini mi bağlayalım, yoksa kendi **minimal Lisp yorumlayıcınızı** yazmayı mı hedefliyorsunuz?

Tercihinize göre ilk **"Hello World" TUI ve Lisp binding** kod taslağını oluşturabilirim.